```
---
title: "Algoritmos"
author: "Alondra Ludueña"
date: "`r Sys.Date()`"
output:
  pdf_document: default
  word_document: default
  html_document: default
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for
authoring HTML, PDF, and MS Word documents. For more details on using R
Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes
both content as well as the output of any embedded R code chunks within the
document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a MatLab que trabaja con variables que en general
pueden verse como matrices.

```{r}
a=38
A=40
a=40
b=60
c=b-a
c
```

```{r}
y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

Exploración de Datos Preinstalados

R incluye una librería de bases de datos para práctica. Para ver el catálogo de
opciones disponibles, se utiliza el comando:

-   data(): Abre una lista de todos los datasets internos que puedes cargar.

Visualización y Acceso a la Información
```

Cuando trabajamos con un dataset específico, como cars (datos sobre velocidad y distancia de frenado), tenemos dos formas principales de interactuar con él:

- Llamada Global (cars): Escribir el nombre del objeto muestra la estructura completa de la base de datos (filas y columnas). Es útil para entender la relación entre las variables.

- Acceso por Columna (cars\$speed): El símbolo \$ actúa como un filtro o selector. Permite extraer una sola columna específica del dataset para trabajar con ella como un vector independiente.

```{r cars}
summary(cars)
```

Including Plots

You can also embed plots, for example:

```{r pressure}
plot(pressure,main="Presión del Gas Ideal",ylab="Presión [hPa]",xlab="Temperatura [K]",type="b", col="purple")
```

Comando plot

Es un comando que permite graficar cualquier fuente de datos que le asignamos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al gráfico. En caso de duda de como utilizar el comando podemos escribir ##?plot## en la consola, lo cual abrirá una ventana que explicará a detalle el comando.

Funciones estadísticas

```{r}
z1<-rnorm(350,21,5)
z1
```

```{r}
plot(z1,col="darkblue") #comentario
w1<-length(z1)
w1
x1<-(500:849)
x1
plot(x1,z1,col="darkorange")
lines(x1,z1,col="brown",type="h")
```

```{r}
hist(z1, main="Histograma de edades",breaks=20,col="pink")
```

```{r}
density(z1)
```

```
```

```{r}
plot(density(z1),col="darkred",type="b")
```

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicios de algoritmos
Las últimas 3 cifras de mi DNI son 848

Ejercicio 1
Crear una variable que tenga ese número

```{r}
dni=848
```
Ejercicio 2
Crear un vector que tenga todos los números desde el 1 hasta el 848 enteros

```{r}
secuencia_dni<-(1:dni)
secuencia_dni
```

Ejercicio 3
Calcular la suma de todos los valores del vector secuencia_dni

```{r}
sum(1:length(secuencia_dni))
```

Ejercicio 4
Repetir el ejercicio anterior en Phyton

```{python}
secuencia_dni=range(1,848)
total=sum(secuencia_dni)
print(total)
```

Ejercicio 5
Repetir el ejercicio anterior con Julia.
No se programó para no ocupar memoria.

Ejercicio 6
Determinar CPU time used.
```{r}
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```
```{r}
require(stats)
system.time(for(i in 1:length(secuencia_dni)){})
```

```
total<-0
valorfinal<-length(secuencia_dni)
for (i in 1:valorfinal){
total<-total+secuencia_dni[i]
}
print(total)
```
```{r}
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

Ejercicio 7
¿Cuánto tarda en correr el código del ejercicio 2? con system.time

```{r}
tiempo <- system.time({
 sum(1:length(secuencia_dni))
})
print(tiempo)
```

Bibliotecta tictoc
Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando install.packages() o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones tic y toc son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluacion comparativa que el tiempo de sistema recién demostrado. Sin embargo, tictoc agrega mucha más comodidad al usuario y armonía al conjunto.

```{r}
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
## [1] "...suena el despertador"
toc()
```

Biblioteca rbenchmark
La documentación de la función benchmark del paquete rbenchmark R lo describe como un simple contenedor alrededor de system.time. Sin embargo, agrega mucha conveniencia en comparación con las llamadas simples a system.time. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

Implementación de una serie Fibonachi o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales:0,1,1,2,3,5,8 ... 89,144,233 ...
La sucesión comienza con los números 0 y 1,2 a partir de estos, cada término es la suma de los dos anteriores, es la relación de recurrencia que la define. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca Universidad de Florencia. Liber Abachi- Autor Fibonacci

Ejercicio 8
¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```{r}
# Inicialización
a <- 0
b <- 1
serie <- c(a, b)  # guardamos la serie
contador <- 2     # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
siguiente <- a + b
a <- b
b <- siguiente
serie <- c(serie, b)  # agregamos a la serie
contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
cat("Primer número mayor a 20:", b, "\n")
cat("Serie de Fibonacci hasta ese número:\n")
print(serie)
```

Ejercicio 9
Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```{r}
#### Instalar paquete
if (!require(microbenchmark)) {
install.packages("microbenchmark")
library(microbenchmark)
}
#### Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
#### Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
n <- length(x)
for (i in 1:(n-1)) {
for (j in 1:(n-i)) {
if (x[j] > x[j+1]) {
```

```
temp <- x[j]
x[j] <- x[j+1]
x[j+1] <- temp
}
}
}
return(x)
}
#### Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
burbuja = bubble_sort(vector),
sort_R = sort(vector),
times = 5
)
print(resultado)
```

La penitencia de Newton
Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto improcedente lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

Ejercicio 10 Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1*1010 y verifica cuál de los dos es más eficiente.

```{r}
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
suma1 <- suma1 + i
}
suma1
#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({
suma1 <- 0
for (i in 1:1010){
suma1 <- suma1 + i
}
})
# Método 2
system.time({
suma2 <- 1010 * (1010 + 1) / 2
```

```
})
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1*1010. El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente